OS HW 1 — Part A: Theory

Ariel Arusy Nate Kinzelberg

TQ1 — Concurrency vs. Parallelism

A program has 3 threads that each need 40 ms of CPU time. There is no I/O and no blocking.

1. On a single core, assuming the system switches between runnable threads and context-switch cost is negligible, what is the minimum wall-clock time until all threads finish?

   Answer: Because context-switch cost is negligible, the core runs all three threads sequentially without any wasted time. The minimum wall-clock time is therefore

   $$3 \times 40\,\text{ms} = \boxed{120\,\text{ms}}.$$

2. On a 3-core CPU, same assumptions, what is the minimum wall-clock time?

   Answer: With three cores available, all three threads can execute in parallel. The minimum wall-clock time is therefore

   $$\left\lceil \frac{3}{3} \right\rceil \times 40\,\text{ms} = \boxed{40\,\text{ms}}.$$

3. If each context switch costs 0.2 ms, and on the single core the system forces a context switch after every 5 ms of CPU execution of the currently running thread, what is the extra overhead added purely due to context switching until completion? Assume a context switch also occurs after the final 5 ms chunk that completes the last thread.

   Answer: Each thread requires 40 ms of CPU time, divided into

   $$\frac{40\,\text{ms}}{5\,\text{ms}} = 8 \text{ chunks per thread.}$$

   Across all three threads this gives $3 \times 8 = 24$ chunks total. A context switch occurs after each chunk (including the final one), yielding 24 context switches. The extra overhead due purely to context switching is

   $$24 \times 0.2\,\text{ms} = \boxed{4.8\,\text{ms}}.$$

TQ2 — Interleavings

A shared/global integer x starts at 0. Two threads, A and B, run concurrently the following code **once each**:

```
t = x
t = t + 1
x = t
```

Assume t is a **thread-local** variable (each thread has its own t).

1. What final values of x are possible?

   Answer: The possible final values are $x \in \{1, 2\}$.

2. Give an explicit interleaving that leads to each possible final value.

   Answer: Let `a.t` and `b.t` denote the thread-local copies of t for threads A and B.

   Case $x = 2$:

   $$\texttt{a.t} \leftarrow 0 \;\rightarrow\; \texttt{a.t} \leftarrow 1 \;\rightarrow\; x \leftarrow 1 \;\rightarrow\; [\text{switch}] \;\rightarrow\; \texttt{b.t} \leftarrow 1 \;\rightarrow\; \texttt{b.t} \leftarrow 2 \;\rightarrow\; x \leftarrow 2.$$

   Case $x = 1$ (lost update):

   $$\texttt{a.t} \leftarrow 0 \;\rightarrow\; \texttt{a.t} \leftarrow 1 \;\rightarrow\; [\text{switch}] \;\rightarrow\; \texttt{b.t} \leftarrow 0 \;\rightarrow\; \texttt{b.t} \leftarrow 1 \;\rightarrow\; [\text{switch}] \;\rightarrow\; x \leftarrow 1 \;\rightarrow\; x \leftarrow 1.$$

   Both threads read $x = 0$ before either writes back, so both write 1 and the second increment is lost.

TQ3 — TAS vs. Ticket Lock

Consider the following two locks: a **test-and-set (TAS) spinlock**, and a **ticket lock** where each thread gets `my_ticket = fetch_add(ticket,1)` and waits for `cur_ticket == my_ticket`.

1. Which lock guarantees **FIFO acquisition order**? Prove it.

   Answer: The **TAS spinlock does not** guarantee FIFO acquisition order. The **ticket lock does**.

   Proof. *Suppose thread A arrives before thread B, so A receives ticket i and B receives ticket j, with $i < j$ (since **`fetch_add`** is atomic, arrival order maps directly to ticket order).*

   *Assume for contradiction that B acquires the lock before A, meaning **`cur_ticket`** reaches j while A has not yet acquired the lock.*

   *By the release code, **`cur_ticket`** starts at 0 and is only ever incremented by 1, atomically, once per release. For **`cur_ticket`** to equal j, it must have advanced through*

   $$0 \;\rightarrow\; 1 \;\rightarrow\; \cdots \;\rightarrow\; i \;\rightarrow\; \cdots \;\rightarrow\; j.$$

In particular it must have passed through i (since $i < j$), meaning the thread with ticket i already acquired and released the lock. But ticket i belongs to A — so A already ran. Contradiction. □

2. Which lock guarantees **bounded waiting** (no starvation), assuming every thread that acquires the lock eventually releases it? Prove it.

   Answer: The **ticket lock** guarantees bounded waiting.

   Proof. *Suppose a thread arrives and atomically acquires ticket number k via* `fetch_add`*. At this moment let* `cur_ticket` $= c$*, so the thread currently holding (or next to hold) the lock has ticket c. Since $c \leq k$, and by the assumption that every thread that acquires the lock eventually releases it,* `cur_ticket` *will increment through*

   $$c \;\to\; c+1 \;\to\; \cdots \;\to\; k-1 \;\to\; k.$$

   By part 1, any thread arriving after this one receives a ticket greater than k and cannot acquire the lock first. It follows that thread k enters its critical section after exactly $k - c$ threads — those holding tickets $c, c+1, \ldots, k-1$ — have each released the lock. This is a finite, fixed bound determined at arrival time, so no thread can starve. □

3. Give a short argument (one paragraph) why TAS can starve a thread.

   Answer: Consider the following scenario. Thread A is waiting while thread B holds the lock. When B releases, A and a brand-new thread C race on `test_and_set` simultaneously — C wins. When C releases, another new thread D arrives and beats A again. This can repeat indefinitely: A keeps losing the race to freshly arriving threads and never acquires the lock, despite waiting the entire time. Because TAS assigns no position or priority to waiting threads — every `test_and_set` is an unordered race regardless of how long a thread has been spinning — there is no bound on how many threads can overtake a waiting thread, and starvation is possible.

TQ4 — Producer/Consumer

We have a bounded buffer (queue) of capacity `CAP`, shared by multiple producers and consumers. Bounded semaphore value is always in $[0, \texttt{CAP}]$; unbounded semaphore has no fixed upper bound.

We use three synchronization objects:

- `mutex` — mutual exclusion for the buffer
- `empty` — a **bounded semaphore** with value always in $[0, CAP]$, initialized to `CAP` (number of empty slots)
- `full` — a **bounded semaphore** with value always in $[0, CAP]$, initialized to `0` (number of full slots)

A student wrote the following high-level algorithm:

Producer (buggy version)

1. `acquire mutex`
2. `wait(empty)`
3. `insert one item into the buffer`
4. `signal(full)`
5. `release mutex`

Consumer (buggy version)

1. `acquire mutex`
2. `wait(full)`
3. `remove one item from the buffer`
4. `signal(empty)`
5. `release mutex`

1. **Explain why this algorithm can deadlock. Your explanation must include a concrete execution scenario (who holds what and who is waiting for what).**

   Answer: Assume the buffer is full (`empty` = 0, `full` = `CAP`).

   1. The producer calls `acquire mutex` — succeeds, mutex is now 0. The producer holds the mutex.
   2. The producer calls `wait(empty)` — since the buffer is full, `empty` = 0, so the producer blocks here while still holding the mutex.
   3. The consumer calls `acquire mutex` — mutex is already 0, so the consumer blocks waiting for the mutex.

   At this point: the **producer** holds the mutex and is waiting for the consumer to free a slot; the **consumer** is waiting for the mutex to be released so it can free a slot. Neither can proceed, this is a deadlock. The symmetric case occurs when the buffer is empty: the consumer grabs the mutex, blocks on `wait(full)`, and the producer then blocks trying to acquire the mutex.

2. **Give the minimal fix (change as little as possible) to make the algorithm deadlock-free, and explain why your fix works.**

   Answer: Swap steps 1 and 2 in both the producer and consumer.

   ## Producer (fixed)

   1. `wait(empty)`
   2. `acquire mutex`

3. `insert one item into the buffer`
4. `signal(full)`
5. `release mutex`

Consumer (fixed)

1. `wait(full)`
2. `acquire mutex`
3. `remove one item from the buffer`
4. `signal(empty)`
5. `release mutex`

This works because a thread now blocks on the semaphore *before* acquiring the mutex. A thread only holds the mutex when it is guaranteed to complete its buffer operation immediately, no blocking while holding the lock is possible, so deadlock cannot occur.